C# Advanced

Les tableaux C#

- Une suite d'élément du **même Type** de **taille fixe :**

```
//Déclaration
int[] monTableau = new int[5];

//mise à jour de valeur
monTableau[0] = 42;

//accès à une valeur
int valeur = monTableau[3];
```

- Il est possible de l'initialiser de différentes façons :

```
int[] superTableau = new int[3] { 1, 2, 3 };
int[] superTableau2 = new int[] { 1, 2, 3 };
int[] superTableau3 =  { 1, 2, 3 };
```

Les tableaux - manipulations

- Le framework .NET facilite la manipulation des tableaux :

```
//Déclaration
int[] monTableau = new int[5];

//longueur du tableau
int nombreDElement = monTableau.Length;

//extraction d'une sous partie
int[] sousTableau = monTableau[1..3];

//tri du tableau grâce à la classe Array
Array.Sort(monTableau);
```

Tableaux complexes

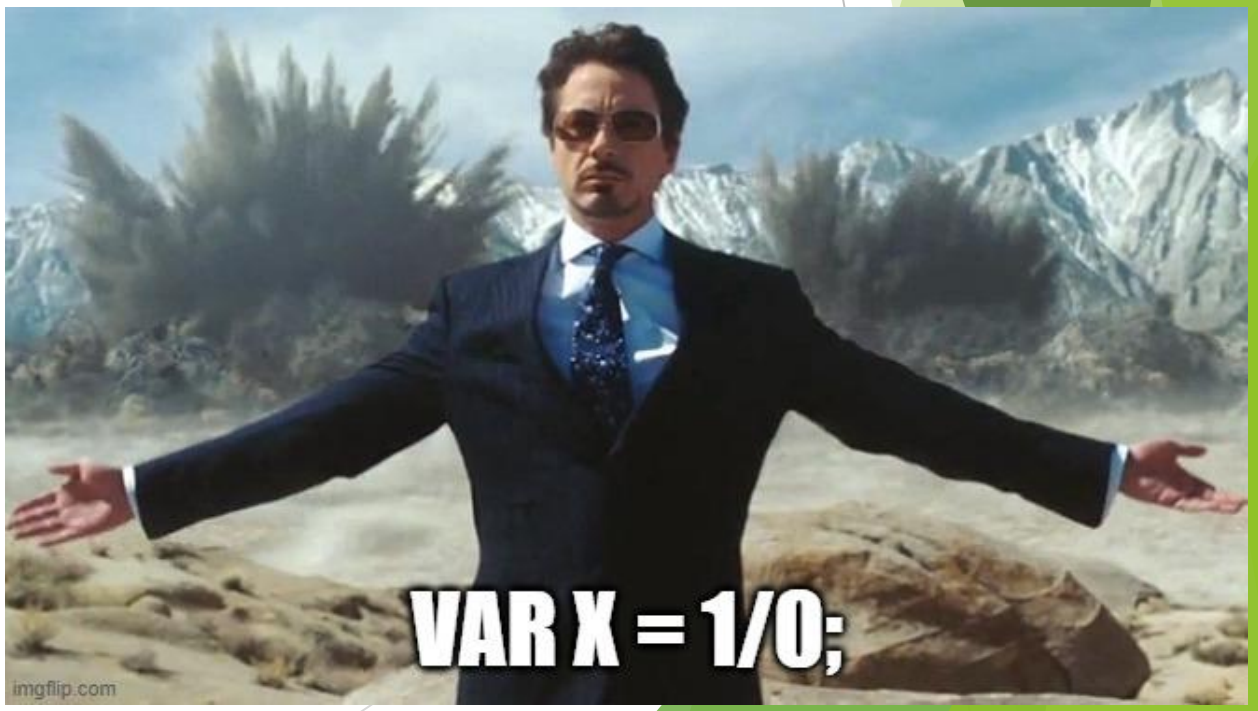
```
//tableaux imbriqués
int[][] nestedArray = new int[5][];
nestedArray[0] = [1, 3, 5];
nestedArray[1] = [10, 20, 30, 40, 50, 60];
nestedArray[2] = [100];

nestedArray[1][2] = 33;

int[][] complex =
    [
    [1, 3, 5],
    new int[] {10, 20, 30, 40, 50, 60 },
    [100]
    ];
```

Les exceptions

- C# est un langage managé ce qui veut dire que les erreurs ne peuvent pas altéré la mémoire d'un autre programme (sauf pour le code « unsafe »)
- Lorsqu'une erreur survient, une exception est lancée stoppant le déroulement du programme
- Exemple d'erreur :
  - Lecture d'un fichier inexistant
  - Division par zéro
  - Tentative d'accéder à une propriété d'une instance null
  - Tentative d'accéder à un index en dehors des limites du tableau
  - Etc.



Exercice

- Récupérez le programme suivant : divinaet/epsi2026 (github)
- Lancez le programme, jouez avec :
  - Essayez 3
  - Essayez 15
  - Essayez toto

- Que se passe-t-il ?
- Quels informations sont à ma disposition ?

Comment gérer les exceptions

- Le bloc try{ … } catch { … } permet de récupérer programatiquement l'exception et de gérer par le code l'erreur
- Un catch par type d'exception
- Finally est exécuté qu'il y ai une erreur ou non

```
try
{
    //Code
}
catch(DivideByZeroException dbe)
{
    //Code
}
catch (Exception e)
{
    //Code
}
finally
{
    //Code
}
```

```
try
{
    //Code
}
finally
{
    //Code
}
```

Mauvais pratique

- Cacher une exception et faire comme si tout aller bien peut mener à
  - Corrompre une base de données
  - Effectuer des traitements avec des mauvaises données
  - Laisser penser qu'un traitement s'est bien déroulé (écriture dans un fichier, envoi d'un mail, etc.) alors que cela ne s'est pas passé comme prévu !

```
1 référence | 0 modification | 0 auteur, 0 modification
public string GetElement(int index)
{
    try
    {
        return this._chaines[index];
    }
    catch(Exception e)
    {
        return "tout va bien !";
    }
}
```

Exceptions courantes

- InvalidArgumentException
- NullReferenceException
- IndexOutOfRangeException
- FileNotFoundException

Bonnes pratiques

- La classe Exception peut être dérivée pour créer une exception plus « fonctionnelle »
  - Les autres exceptions doivent continuées à arrêter le programme pour ne pas risquer d'empirer la situation

```
public class ExceptionFunctionnelle : Exception
{
    1 référence | 0 modification | 0 auteur, 0 modification
    public int DonneeSupplementairePourAnalyse { get; set; }

    0 références | 0 modification | 0 auteur, 0 modification
    public ExceptionFunctionnelle(string message) : base(message) { }
    0 références | 0 modification | 0 auteur, 0 modification
    public ExceptionFunctionnelle(int data, string message) : base(message)
    {
        this.DonneeSupplementairePourAnalyse = data;
    }
}
```

Bonnes pratiques

- En cas de détection de problème, l'exception fonctionnelle est renvoyée

```
1 référence | 0 modification | 0 auteur, 0 modification
public string GetElement(int index)
{
    if (index > 10)
    {
        throw new ExceptionFunctionnelle(index, "index invalide");
    }
    return _chaines[index];
}
```

- Une exception fonctionnelle peut permettre de rendre plus lisible une exception sous jacente, dans ce cas on l'intègre (InnerException) :

```
1 référence | 0 modification | 0 auteur, 0 modification
public string GetElement(int index)
{
    try
    {
        return this._chaines[index];
    }
    catch(IndexOutOfRangeException e)
    {
        throw new ExceptionFunctionnelle(index, "Index invalide !", e);
    }
}
```

Bonnes pratiques

- Dans le cas des bibliothèques il est possible de préciser dans la documentation les exceptions auxquelles le développeur peut s'attendre :

```
/// <summary>
/// Renvoi l'élément du tableau selon l'index
/// </summary>
/// <param name="index">entier de 0 à 9</param>
/// <returns></returns>
/// <exception cref="ExceptionFunctionnelle">Si l'index est plus grand que 10
/// cette exception est envoyée</exception>
1 référence | 0 modification | 0 auteur, 0 modification
public string GetElement(int index)
{
    try
    {
        return _chaines[index];
    }
    catch(IndexOutOfRangeException ex)
    {
        throw new ExceptionFunctionnelle(index, ex.Message);
    }
}
```

Bonnes pratiques

- Dans le programme appelant on gère l'exception et le programme peut continuer :

```
try
{
    string chaine = _worker.GetElement(index);
    Console.WriteLine(chaine);
}
catch(ExceptionFunctionnelle e)
{
    Console.WriteLine($"erreur lors du traitenement : {e.Message}");
}
```

Exercice

- Prévenir les problèmes avec du TDD
  - Un attribut supplémentaire à TestMethod permet d'indiquer qu'on attend une exception:

```
{
    [TestMethod()]
    [ExpectedException(typeof(ArgumentException))]
    0 références | 0 modification | 0 auteur, 0 modification
    public void GetElementTest()
```

- Créer une Exception « IndexTooLongException »
  - Lancer cette exception dans Worker si l'entier est supérieur à 10
  - Catcher cette exception dans Program.cs pour dire à l'utilisateur que son entier n'est pas bon

- Créer une Exception « InvalidStringException »
  - Lancer cette exception dans Worker si la chaîne n'est pas un entier
  - Catcher cette exception dans Program.cs pour avertir l'utilisateur

Héritage - généralisation

- A la création d’une classe, celle-ci dérive automatiquement de « Object »
  - Il est possible de surcharger « ToString() »
  - C’est la méthode appelée pour la convertion implicite vers String !
- Rappel surcharge :
  - La classe parente doit déclarer la méthode comme « virtual »
  - La classe enfant doit surcharger en précisant « override »
- Question : que se passe-t-il si un des deux éléments est manquant ?

Généricité

- Possibilité de définir des comportement « générique », indépendamment des éléments sous jacents

  Exemple : gestion d'un album photo, album de timbre, album de carte postale…
  Il faudrait une classe Album spécifique pour chaque élément

- Une classe générique se définit en utilisation le diamant : Album<T>

```
public class Album<T>
{
    T[] arrayOfItems;

    public Album(int size)
    {
        this.arrayOfItems = new T[size];
    }
    public void AddItem(T item) { }
    public void UpdateItem(int index, T item) { }
    public void RemoveItem(T item) { }
    public T RemoveItem(int index) { /* return... */ }
}
```

Généricité - utilisation

- Pour utiliser la classe, il faut remplacer T par un type concret :

```
Album<Music> albumOfMusics = new Album<Music>(10);
albumOfMusics.AddItem(new Music());
albumOfMusics.UpdateItem(0, new Music());
Music music = albumOfMusics.RemoveItem(0);

Album<Photo> albumOfPhotos = new Album<Photo>(5);
albumOfPhotos.AddItem(new Photo());
albumOfPhotos.UpdateItem(0, new Photo());
Photo photo = albumOfPhotos.RemoveItem(0);
```

Généricité - contraintes

- Il est possible de restreinte les possibilités au niveau de T (quel type de base ou classe il est possible d'utiliser)

where T : class	T est une classe (et non un type valeur)
where T : struct	T est un type valeur (et non une classe)
where T : new()	T possède un constructeur par défaut
where T : « xyz »	T hérite de la classe « xyz »
where T : ICloneable	T implémente une interface

```
public class Album<T>
    where T : class, new()
{
    ...
}
```